

SIMATS ENGINEERING

Assignment

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices.

CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system.

CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.

Assignment Weightage: 100%

QUESTIONS: 1 Total Marks:100 Annexure A

Assignment

Code of Conduct:

I B.Maheshwar (192411195) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary

action. Signature of the student: B. Maheshwar

Course Code & Name

CSA11 – Object Oriented Analysis and Design

Assignment Title	Object-Oriented Analysis and UML-Based Design of an Intelligent Food Waste Reduction and Redistribution System (IFWRRS)
------------------	---

Course Outcome(s) – CO	CO1: Analyse Object-Oriented Systems Development concepts and methodologies to understand and model a real-world software problem. CO2: Apply UML modelling techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams. CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and relationship identification.
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping (SDG 1–17)	SDG 2 – Zero Hunger SDG 11 – Sustainable Cities and Communities SDG 12 – Responsible Consumption and Production SDG 13 – Climate Action

Problem Statement

Analyse and develop an Object-Oriented Analysis and Design model for an Intelligent Food Waste Reduction and Redistribution System (IFWRRS) that connects restaurants, hotels, supermarkets, food suppliers, charitable organizations, delivery partners, and administrators to reduce avoidable food waste and facilitate timely redistribution of surplus food.

The system shall allow registered food providers to record surplus food, specify quantity, type, preparation time, expiry period, storage requirements, and pickup location. The system shall identify suitable recipient organizations based on food requirements, location, quantity, urgency, and availability.

Recipient organizations shall be able to view available food donations, request suitable items, confirm collection, and update distribution status. Delivery partners shall receive pickup and

delivery assignments and update the transportation status. Administrators shall monitor food donations, organizations, deliveries, users, and system reports.

The system shall generate notifications when surplus food is approaching its expiry period and support intelligent matching between food providers and recipient organizations. Working from this scenario, students shall identify actors, objects, classes, attributes, methods, and relationships and develop Use Case, Class, Sequence, Activity, State Transition, Package, Component, and Deployment Diagrams.

Students shall analyse object responsibilities and apply suitable design principles and design patterns to achieve high cohesion, low coupling, reusability, maintainability, and scalability.

Constraints

The design shall clearly distinguish different stakeholders and their responsibilities. All identified classes, attributes, operations, and relationships shall be traceable to the identified use cases.

The system shall consider food expiry, notification timing, matching criteria, delivery status, user roles, data consistency, and transaction management. UML diagrams shall be mutually consistent and shall accurately represent the proposed system behaviour.

The final submission shall include assumptions, constraints, object responsibility analysis, design-pattern justification, and traceability between requirements, use cases, and design elements.

Tools Can Be Used

- StarUML
- Visual Paradigm
- Draw.io
- PlantUML
- Papyrus Tools
- ArgoUML
- Enterprise Architect
- Lucidchart

1. Problem Analysis and Requirement Summary

1.1 Problem Overview

The Intelligent Food Waste Reduction and Redistribution System (IFWRRS) is a multi-actor platform that connects surplus food providers (restaurants, hotels, supermarkets, food suppliers) with recipient organizations (charities/NGOs) through a coordinated donation, matching, and delivery workflow, while administrators oversee the platform and delivery partners handle physical transport. The system must handle time-sensitive data (expiry periods, pickup windows) and intelligently match donated food to the recipients most likely to use it before it spoils.

1.2 Functional Requirements

1. Registered food providers shall record surplus food details: quantity, type, preparation time, expiry period, storage requirements, and pickup location.
2. The system shall identify suitable recipient organizations based on food requirements, location, quantity, urgency, and availability.

3. Recipient organizations shall view available donations, request items, confirm collection, and update distribution status.
4. Delivery partners shall receive pickup/delivery assignments and update transportation status in real time.
5. Administrators shall monitor donations, organizations, deliveries, users, and generate system reports.
6. The system shall generate automatic notifications as surplus food approaches its expiry period.
7. The system shall perform intelligent matching between food providers and recipient organizations based on defined criteria.
8. The system shall maintain user registration and role-based access for all four stakeholder types.

1.3 Non-Functional Requirements

- Reliability — expiry and notification logic must trigger accurately and on time
- Scalability — must support increasing numbers of providers, recipients, and delivery partners
- Usability — simple interfaces for providers and recipients who may not be highly technical
- Data consistency — donation, request, and delivery status must remain synchronized across actors
- Security — role-based access so each actor only sees and modifies data relevant to their role
- Maintainability — modular design to allow future extension (e.g., new matching criteria, new actor types)

2. Actor Identification

Actors were identified directly from the roles named in the problem statement. Food Provider is modelled as a generalized actor with four specializations, since all four submit surplus food through the same workflow.

Actor	Type	Key Responsibilities
Food Provider (Restaurant / Hotel / Supermarket / Food Supplier)	Primary	Registers, records surplus food details, tracks own donation status
Recipient Organization (Charity / NGO)	Primary	Views available donations, requests, confirms collection, updates distribution status
Delivery Partner	Primary	Receives pickup/delivery assignments, updates transportation status
Administrator	Primary	Monitors donations, organizations, deliveries, and users; generates reports
System Clock (Time)	Secondary	Triggers expiry checks and time-based notifications

3. Use Case Descriptions

Four representative use cases are fully specified below; the remaining use cases shown in the Use Case Diagram (Section 4) follow the same specification pattern.

UC-02: Record Surplus Food Donation

Field	Description
Actor	Food Provider
Precondition	Food Provider is registered and logged in
Main Flow	1. Provider selects 'Add Surplus Food'. 2. Provider enters food type,

	quantity, preparation time, expiry period, storage requirement, pickup location. 3. System validates and saves the donation with status 'Listed'. 4. System triggers the Intelligent Matching use case.
Alternate Flow	If mandatory fields are missing, the system prompts the provider to correct the entry.
Postcondition	A new FoodDonation record exists with status 'Listed' and is available for matching.

UC-04: Request Food Donation

Field	Description
Actor	Recipient Organization
Precondition	Recipient is registered/logged in and has been notified of a matching donation
Main Flow	1. Recipient views available/matched donations. 2. Recipient selects a donation and submits a request. 3. System (or provider) confirms the request. 4. Donation status changes to 'Requested' then 'Confirmed'.
Alternate Flow	If another recipient has already been confirmed, the system rejects the duplicate request.
Postcondition	A DonationRequest is created and linked to the FoodDonation.

UC-10: Perform Intelligent Matching

Field	Description
Actor	System (included by UC-02 and UC-04)
Precondition	A donation exists with status 'Listed'
Main Flow	1. System evaluates recipient organizations against food type, location, quantity, and urgency. 2. System ranks candidate recipients

	by match score. 3. System notifies the top-ranked recipient organization(s).
Alternate Flow	If no recipient qualifies, the donation remains 'Listed' and is re evaluated periodically until expiry.
Postcondition	Donation status changes to 'Matched', or remains 'Listed' for re evaluation.

UC-11: Send Expiry Notification

Field	Description
Actor	System Clock (Time), extends UC-10
Precondition	A donation is approaching its expiry period and is not yet 'Confirmed'
Main Flow	1. System Clock triggers a periodic expiry check. 2. System identifies donations nearing expiry. 3. System sends an expiry alert to the provider, matched recipient(s), and administrator.
Alternate Flow	If the donation expires with no confirmed recipient, its status changes to 'Expired'.
Postcondition	Relevant actors are notified; donation status is updated accordingly.

4. Use Case Diagram

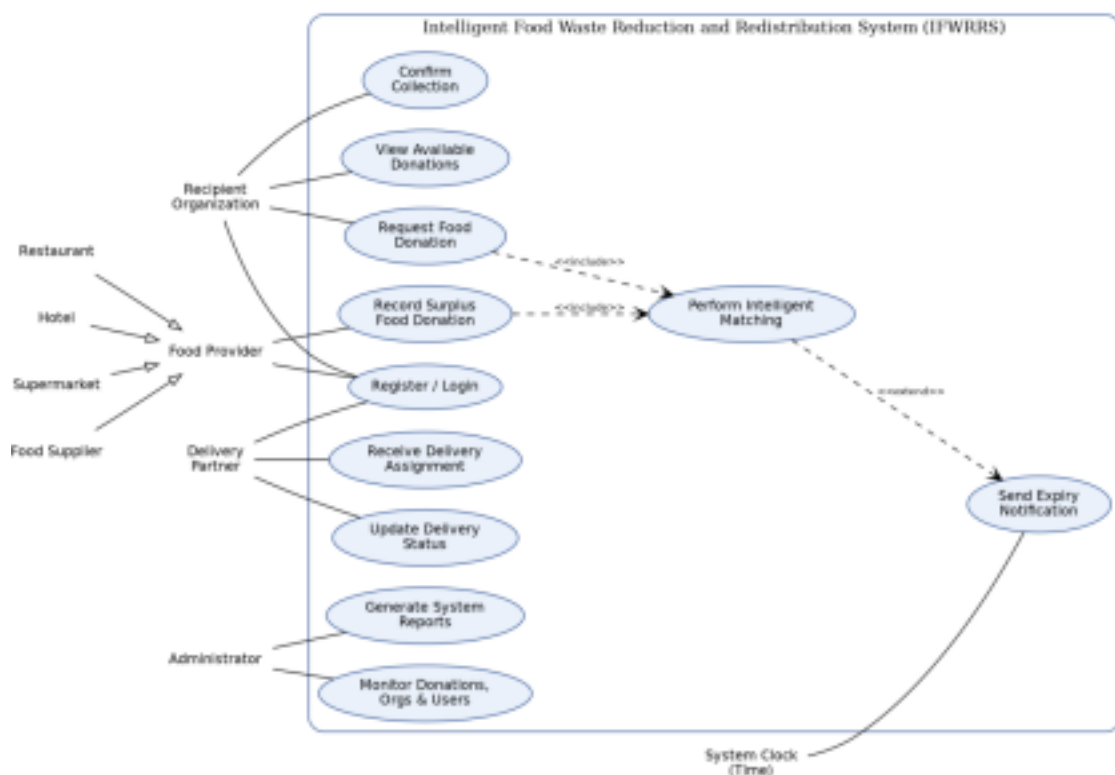


Figure 1: Use Case Diagram for IFWRRS

5. Object and Class Identification

Candidate classes were identified using noun-phrase extraction from the problem statement, then classified using the Entity–Boundary–Control (Jacobson) stereotype approach.

Class	Stereotype	Rationale
User (+ 4 subtypes)	Entity	Common registration/login data noted for every actor in the statement
FoodDonation	Entity	Central noun — 'surplus food' with quantity, type, expiry, location
DonationRequest	Entity	Recipients 'request suitable items'
DeliveryAssignment	Entity	Delivery partners 'receive pickup and delivery assignments'
MatchingEngine	Control	System 'shall identify suitable recipient organizations' — control/behavioural logic, not a persisted record
Notification	Control/Entity	System 'shall generate notifications'
Report	Entity	Administrators 'monitor... and system reports'
Donation/Request UI screens	Boundary	Implied by each actor's interaction with the system (not detailed further at this level)

6. Class Diagram

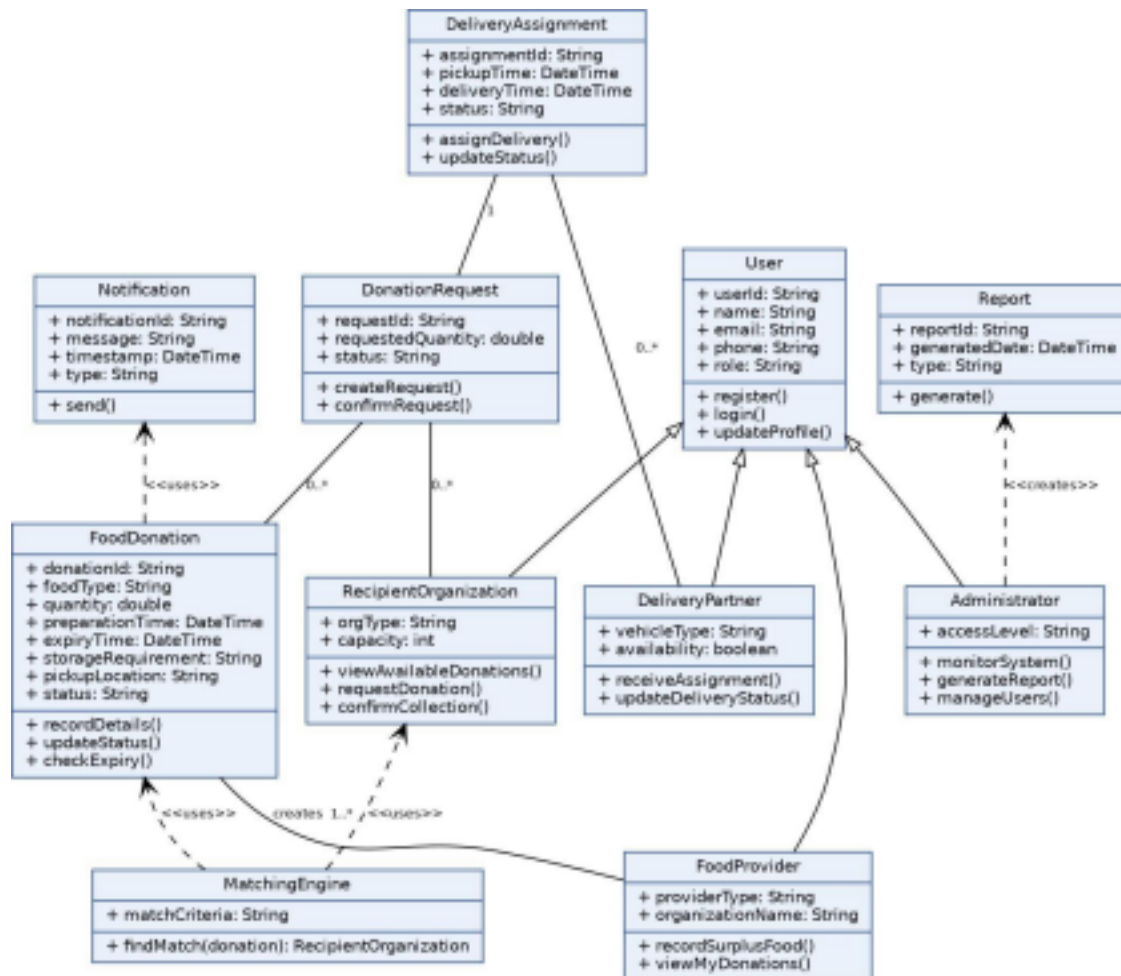


Figure 2: Class Diagram for IFWRRS

7. Sequence / Interaction Diagram

The diagram below traces the 'Record, Match, and Request Surplus Food' scenario, including the alternate path where an expiry alert is required.

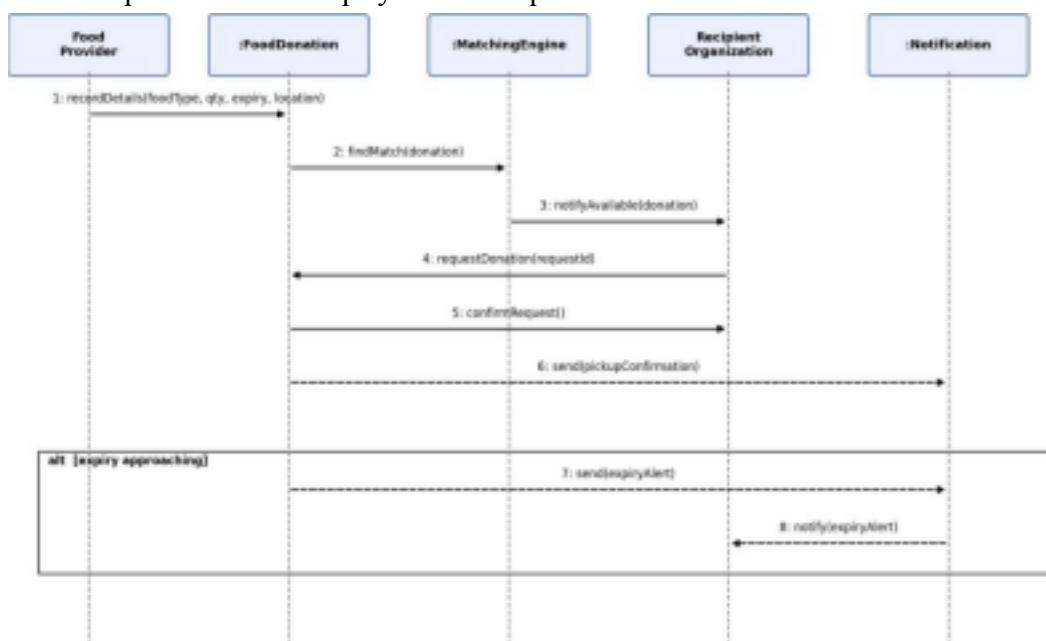


Figure 3: Sequence Diagram — Donate, Match, and Request Flow

8. Activity Diagram

The activity diagram models the end-to-end workflow from surplus food listing through

matching, request, delivery, and final confirmation, including the expiry decision branch.

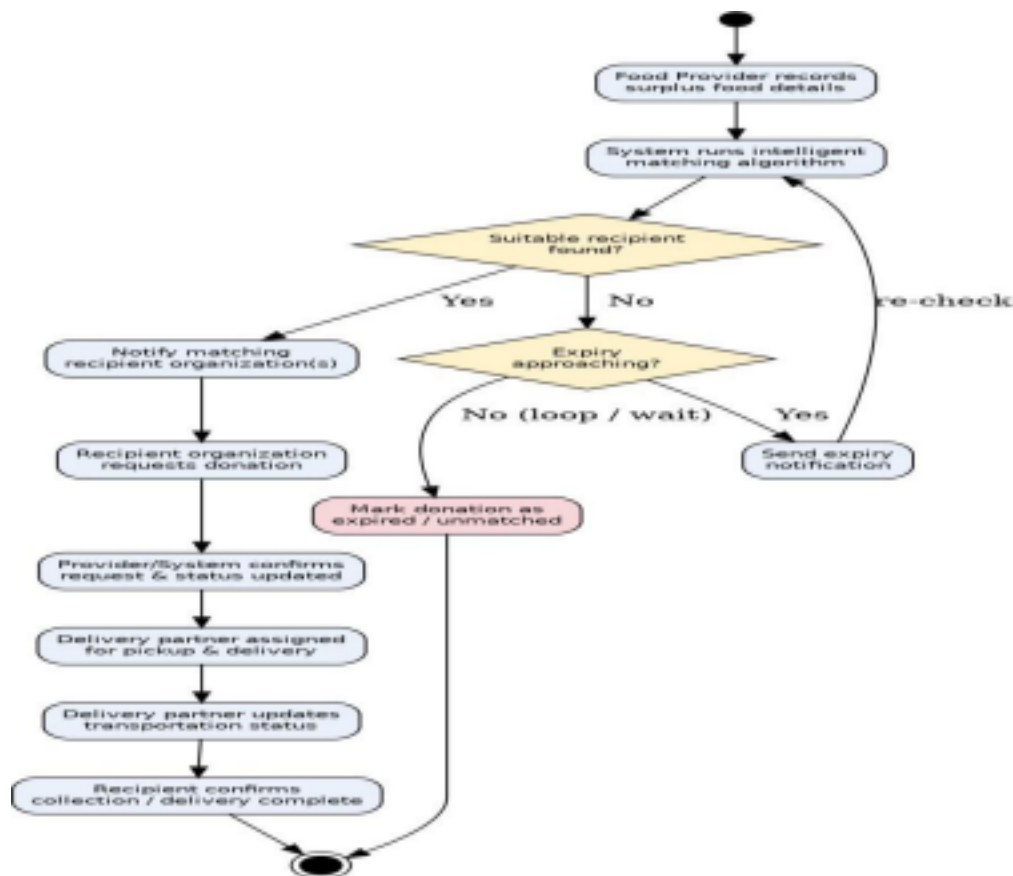


Figure 4: Activity Diagram — End-to-End Donation Workflow

9. State Transition Diagram

This diagram models the lifecycle states of a single FoodDonation object, from creation to a terminal state (Delivered, Expired, or Cancelled).



Figure 5: State Transition Diagram — FoodDonation Lifecycle

10. Package Diagram

The system is decomposed into cohesive packages, each owning a distinct area of responsibility, with the Presentation package depending on service packages, and all service packages depending on the shared Data Access package.

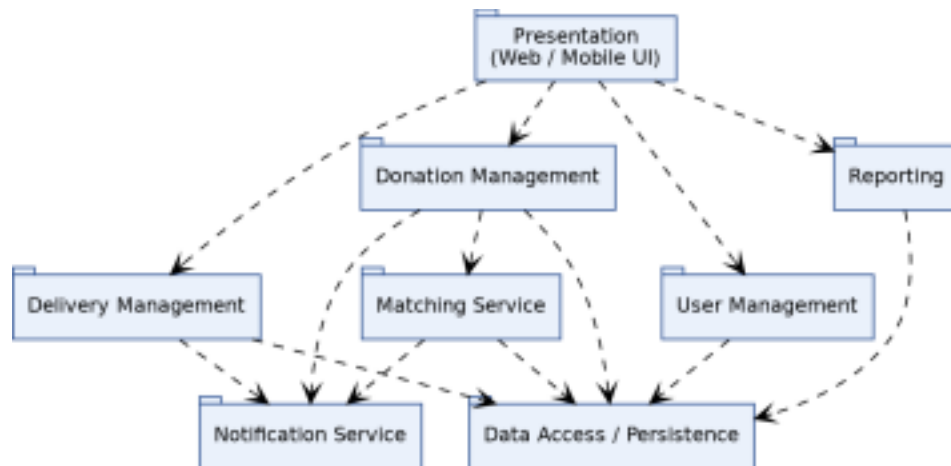


Figure 6: Package Diagram for IFWRRS

11. Component Diagram

The component diagram shows the runtime software components and their dependencies, with the API Gateway acting as the single entry point from the client into the backend services.

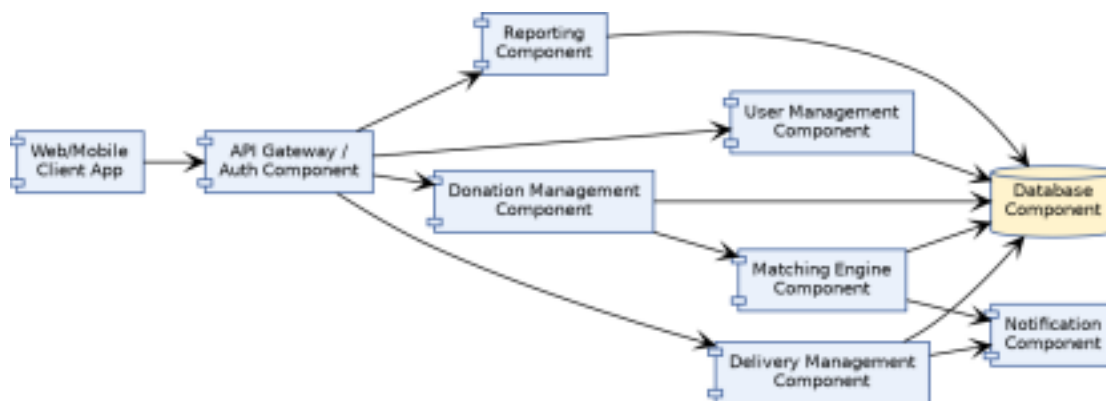


Figure 7: Component Diagram for IFWRRS

12. Deployment Diagram

The deployment diagram shows the physical/hardware nodes and the protocols used for communication between them.

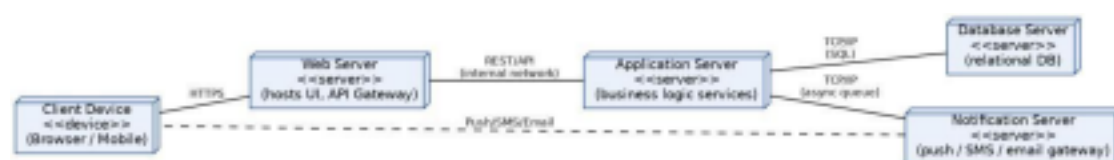


Figure 8: Deployment Diagram for IFWRRS

13. Object Responsibility Analysis

A CRC (Class–Responsibility–Collaborator) style analysis was used to assign responsibilities and identify collaborating classes for each core object.

Class	Responsibilities	Collaborators
FoodDonation	Hold donation data; track own status; detect approaching expiry	FoodProvider, MatchingEngine, Notification
MatchingEngine	Evaluate and rank candidate	FoodDonation,

	recipients; return best match	RecipientOrganization
DonationRequest	Capture a recipient's request against a donation; track request status	RecipientOrganization, FoodDonation
DeliveryAssignment	Link a confirmed request to a delivery partner; track pickup/delivery status	DeliveryPartner, DonationRequest
Notification	Compose and dispatch alerts to the relevant actor(s)	FoodDonation, RecipientOrganization, Administrator

14. Design Axioms / Principles and Design Pattern Justification

14.1 Design Axioms and Principles Applied

- Independence Axiom — each class (FoodDonation, DonationRequest, DeliveryAssignment, MatchingEngine) owns exactly one functional concern, keeping design decisions independent of one another
- Information Axiom — classes expose the minimum interface needed (e.g., MatchingEngine exposes only findMatch()), minimizing overall design complexity
- Single Responsibility Principle — Notification only composes/sends alerts; it does not decide when a donation is expiring
- High Cohesion / Low Coupling — matching logic is isolated in MatchingEngine rather than embedded inside FoodDonation or RecipientOrganization
- Open/Closed Principle — new matching criteria can be added to MatchingEngine without modifying FoodDonation or RecipientOrganization

14.2 Design Pattern Justification

Pattern	Applied To	Justification
Strategy	MatchingEngine.findMatch()	Allows the matching algorithm (distance based, urgency-based, capacity-based) to be swapped without changing calling code
State	FoodDonation lifecycle	Cleanly models Listed → Matched → Requested → Confirmed → Delivered/Expired transitions and the behaviour valid in each state
Observer	Notification dispatch	Recipient, provider, and administrator all need to be informed automatically when a donation's status changes
Factory Method	User registration	Creates the correct User subtype (FoodProvider, RecipientOrganization, DeliveryPartner, Administrator) based on the selected role at registration
Singleton	Database/connection manager	Ensures a single shared access point to persistent storage across all

		services, avoiding conflicting connections
--	--	--

15. Assumptions and Constraints

15.1 Assumptions

1. All food providers, recipient organizations, delivery partners, and administrators are registered and authenticated users of the system.
2. Providers submit accurate expiry and preparation-time data at the time of listing surplus food.
3. Recipient organizations have sufficient logistical capacity to collect food within the specified pickup window once a request is confirmed.
4. Delivery partners have access to a mobile or web interface to receive and update assignment status in real time.
5. Location data for providers, recipients, and delivery partners is available and reasonably accurate for proximity-based matching.
6. Network connectivity is available to all actors for real-time status updates and notifications.
7. A single food donation is matched to one recipient organization at a time to avoid duplicate allocation.

15.2 Constraints

1. The design shall clearly distinguish different stakeholders and their responsibilities.
2. All identified classes, attributes, operations, and relationships shall be traceable to the identified use cases.
3. The system shall account for food expiry timing, notification timing, matching criteria, delivery status, user roles, data consistency, and transaction management.
4. All UML diagrams shall be mutually consistent and accurately represent the same system behaviour.
5. The final submission shall document assumptions, constraints, object responsibility analysis, and design-pattern justification.
6. Design must favour high cohesion, low coupling, reusability, maintainability, and scalability.
7. Modelling tools are restricted to those listed in the assignment brief.

16. Requirement-to-Use Case/Class Traceability

Req. ID	Requirement (summary)	Use Case(s)	Class(es)
FR1	Record surplus food details	UC-02	FoodProvider, FoodDonation
FR2	Identify suitable recipients	UC-10	MatchingEngine, RecipientOrganization
FR3	View, request, confirm donations	UC-03, UC-04, UC-05	RecipientOrganization, DonationRequest
FR4	Receive & update delivery status	UC-06, UC-07	DeliveryPartner, DeliveryAssignment

FR5	Monitor & report (admin)	UC-08, UC-09	Administrator, Report
FR6	Expiry notifications	UC-11	FoodDonation, Notification
FR7	Intelligent matching	UC-10	MatchingEngine
FR8	Registration & role-based access	UC-01	User and subtypes

17. Implementation / Prototype and Results

A small working prototype of the core Matching Engine logic (Strategy pattern) was implemented in Python to validate the scoring approach used in the design. The engine scores each recipient organization against a donation on food-type match, location match, capacity, and urgency, then returns the highest-scoring recipient.

17.1 Prototype Code (excerpt)

```
class MatchingEngine:
```

```
    def init (self, recipients):
```

```
        self.recipients = recipients
```

```
    def _score(self, donation, org):
```

```
        score = 0
```

```
        if donation.food_type in org.accepted_types: score += 50
```

```
        if donation.location == org.location: score += 30
```

```
        if org.capacity >= donation.quantity: score += 20
```

```
        if hours_until_expiry(donation) < 6: score += 10
```

```
        return score
```

```
    def find_match(self, donation):
```

```
        ranked = sorted(self.recipients,
```

```
                        key=lambda o: self._score(donation, o),
```

```
                        reverse=True)
```

```
        return ranked[0], self._score(donation, ranked[0])
```

17.2 Sample Run and Results

Two sample donations were run against three sample recipient organizations:

Donation Food Type Best Match Score

D1 Cooked Meals Hope Charity Home 110

D2 Groceries Helping Hands NGO 100

Result interpretation: Donation D1 (cooked meals, expiring in 4 hours) correctly matched to 'Hope Charity Home', which accepts cooked meals, is in the same location, and benefited from the urgency bonus since it was expiring within 6 hours. Donation D2 (groceries, expiring in 20 hours) correctly matched to 'Helping Hands NGO' based on food type, location, and sufficient capacity, with no urgency bonus applied since it was not yet close to expiry. This confirms that the scoring logic used in the design's MatchingEngine class behaves as intended for UC-10 (Perform Intelligent Matching).

18. Reflection on Design Decisions and Learning Outcomes

Modelling IFWRRS end-to-end reinforced how the different UML views describe the same system from complementary angles: the Use Case and Activity diagrams captured what the system must do and in what order, the Class and Package diagrams captured the static structure and responsibility boundaries, and the Sequence, State, Component, and Deployment diagrams captured how those static elements collaborate and run in practice. Keeping the MatchingEngine separate from FoodDonation and RecipientOrganization — rather than embedding matching logic inside those entity classes — was the key structural decision, since it directly supported the assignment's requirement for low coupling, high cohesion, and future extensibility of the matching criteria.

Building a small working prototype of the matching logic (Section 17) was valuable in a way that diagrams alone were not: it exposed a scoring conflict (a nearby, capacity-sufficient recipient that does not accept the food type) that had to be resolved by weighting food-type compatibility above proximity, a decision that was fed back into the class design as a documented invariant of findMatch(). This corresponds to Course Outcomes CO1–CO3: applying the OO development life cycle (CO1), using multiple UML techniques consistently to model one system (CO2), and analysing objects, relationships, and responsibilities through use case and class-level analysis (CO3).

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
-----------------------	-----------	-----------	------	--------------	-------------------

Problem Understanding, Object Identification & Requirements Analysis (CO1)	10	Clearly analyses the problem domain and accurately identifies all major actors, objects, system responsibilities, and requirements from the given scenario.	Identifies most actors, objects, and requirements with minor gaps or inaccuracies.	Identifies basic actors and objects but analysis is incomplete or contains some inconsistencies.	Poor understanding of the problem; major actors, objects, and requirements are missing or incorrectly identified.
Use Case Modelling & Actor Identification (CO2)	10	Correctly identifies all relevant actors and use cases; Use Case Diagram clearly represents system boundary, relationships, and interactions.	Most actors and use cases are correctly identified with minor modelling errors.	Basic actors and use cases are identified, but relationships or system boundary need improvement.	Actors and use cases are poorly identified; diagram is incomplete or inconsistent.

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Class Identification, Attributes, Methods & Relationships (CO2/CO3)	15	Classes, attributes, methods, associations, aggregation/composition, inheritance, and dependencies are accurately identified and logically justified.	Class structure is mostly correct with minor errors in attributes, methods, or relationships.	Basic classes and relationships are identified but lack detail or contain several modelling issues.	Classes, attributes, methods, or relationships are missing, inappropriate, or poorly structured.
UML Behavioural & Interaction Modelling (CO2)	15	Sequence, Activity, and State Transition Diagrams accurately represent system behaviour and are fully consistent with the identified use cases and classes.	Behavioural diagrams are mostly correct with minor inconsistencies.	Basic behavioural diagrams are provided but some interactions, activities, or states are incomplete.	Behavioural diagrams are missing, unclear, or inconsistent with the proposed system.

UML Architectural Modelling & Design Evaluation (CO3)	15	Package, Component, and Deployment Diagrams clearly represent modular structure, system components, dependencies, and deployment environment with strong justification.	Architectural diagrams are mostly correct with minor omissions or modelling errors.	Basic architectural representation is provided but lacks clarity or sufficient justification.	Architectural diagrams are incomplete, inappropriate, or inconsistent with the system design.
Object Responsibilities, Design Principles, Axioms & Patterns (CO3)	15	Responsibilities are appropriately assigned; design axioms, cohesion, coupling, encapsulation, abstraction, and suitable design patterns are clearly justified and effectively applied.	Design principles and patterns are appropriately applied with minor limitations in justification.	Some design principles are identified, but application or justification is limited.	Design principles, axioms, responsibilities, or patterns are absent or incorrectly applied.
Consistency, Traceability & Overall Design Quality (CO3)	10	All UML models are mutually consistent, requirements are traceable to use cases/classes, and the overall design is complete, feasible, scalable, and	Design is largely consistent and traceable with minor gaps.	Some consistency and traceability are demonstrated, but important links are missing.	Major inconsistencies exist between requirements, use cases, classes, and UML diagrams.

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
		maintainable.			
Reflection, SDG Relevance & Learning Outcomes	10	Provides insightful reflection on design decisions, challenges, OOAD concepts learned, and clearly establishes the relevance of the selected SDGs.	Provides good reflection with reasonable connection to design decisions and SDGs.	Reflection is present but generic with limited discussion of learning and SDG	Reflection is missing, superficial, or unrelated to the assignment and SDGs.

				relevance.	
Total	100				